BECS 22623
Implementation of Numerical Methods using Appropriate software

Department of Mathematics
Faculty of Science
University of Kelaniya

Overview

Overview

- MATLAB's basics
  - Arithmetic calculation
  - Matrix calculations and plotting
  - Script M-files
  - Function files
  - Input and output

What is MATLAB?

MATLAB is the short form of MATrix LABoratory which is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation.

Typical uses in MATLAB?

MATLAB is widely used as a computational tool for academia and Industry, mainly in science and engineering encompassing the fields of physics, chemistry, math and all engineering streams. MATLAB is used in every aspect of computational mathematics for

- Scientific Computing
- Data Analysis
- Visualization

Features of MATLAB

Following are the basic features of MATLAB:

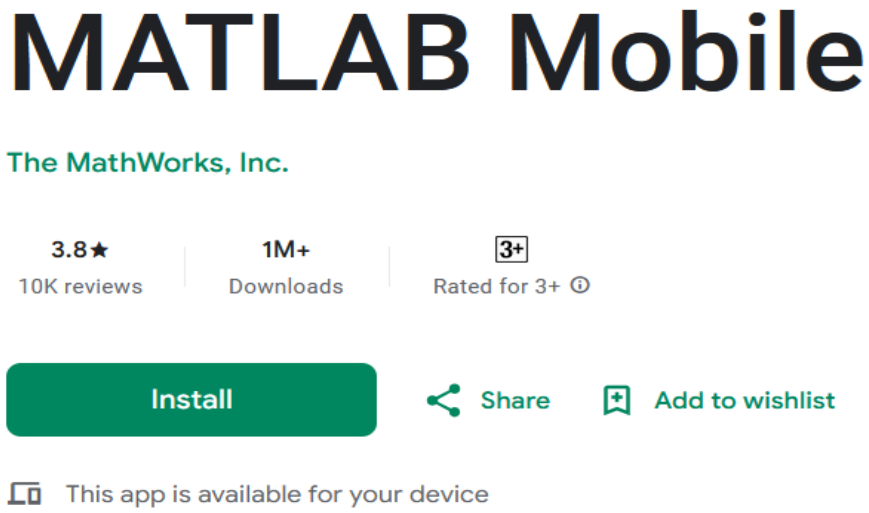
- It is a high-level language for numerical computation, visualization and application development.
- It also provides an interactive environment for iterative exploration, design and problem solving.
- It provides vast library of mathematical functions for linear algebra, statistics, Fourier analysis, filtering, optimization, numerical integration and solving ordinary differential equations.
- It provides built-in graphics for visualizing data and tools for creating custom plots.
- MATLAB's programming interface gives development tools for improving code quality, maintainability, and maximizing performance.
- It provides tools for building applications with custom graphical interfaces.

Starting MATLAB

- You may have an icon set up on your desktop that enables you to start MATLAB with a simple double-click.

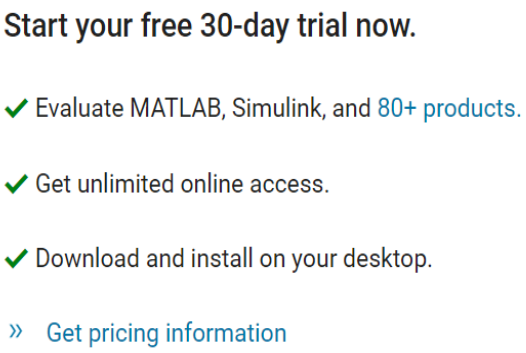
Starting MATLAB

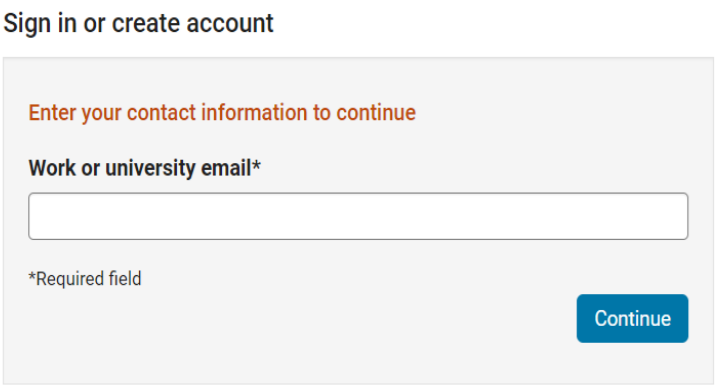
- Mobile version of MATLAB. First download the app and install. Then create a MathWorks account using your email.



Starting MATLAB

- Free MATLAB trail





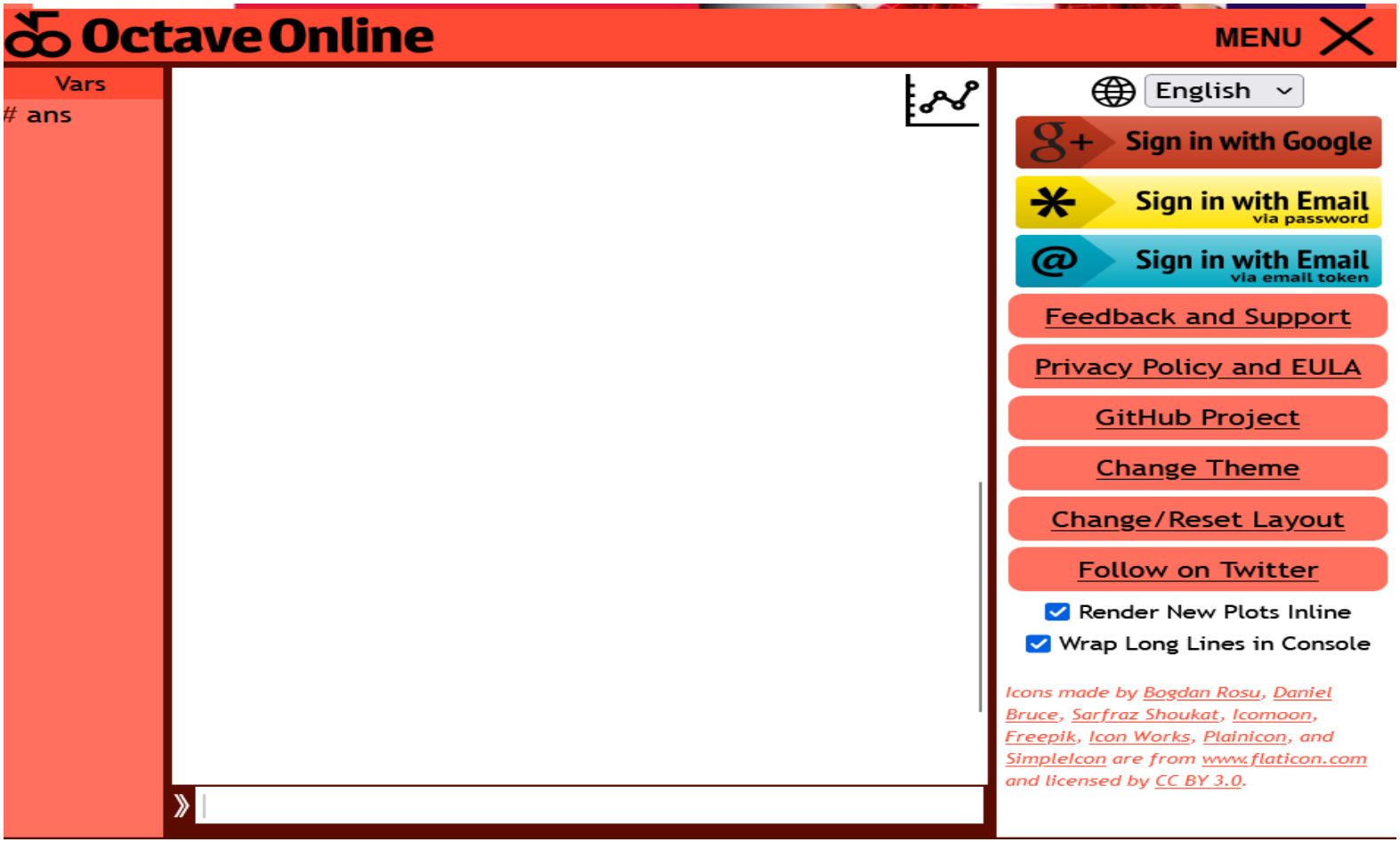
Starting MATLAB

GNU Octave

Scientific Programming Language

- Powerful mathematics-oriented syntax with built-in 2D/3D plotting and visualization tools
- Free software, runs on GNU/Linux, macOS, BSD, and Microsoft Windows
- Drop-in compatible with many Matlab scripts

Starting MATLAB



Understanding the MATLAB Environment

MATLAB development IDE(Integrated Development Environment) can be launched from the icon created on the desktop. The main working window in MATLAB is shown in the desktop. When MATLAB is started, the desktop appears in its default layout:
Then the desktop has the following panels:

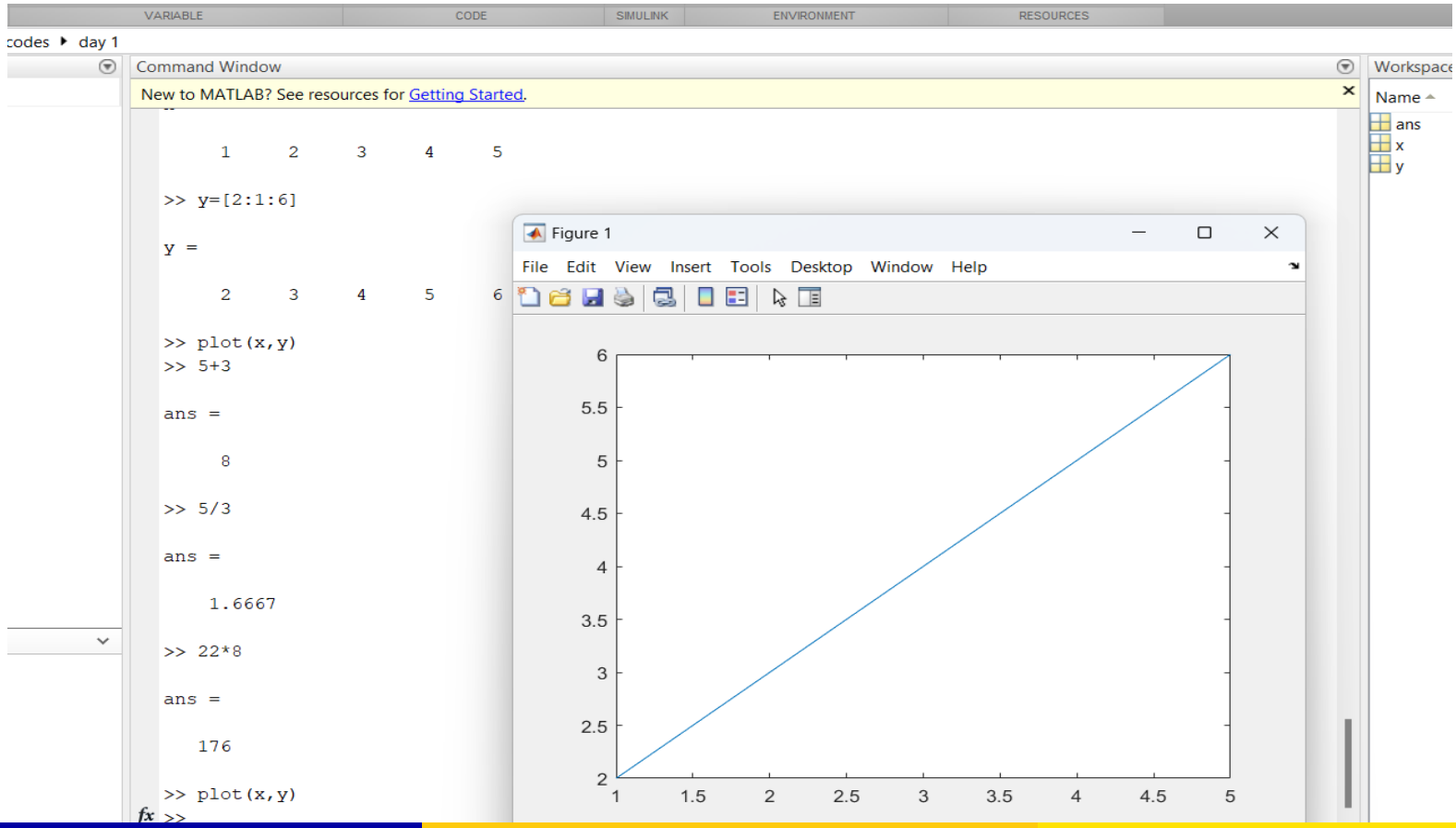
- **Current Folder** - This panel allows you to access the project folders and files.
- **Command Window** - This is the main area where commands can be entered at the command line. It is indicated by the command prompt (>>).
- **Workspace** - The workspace shows all the variables created and/or imported from files.
- **Command History** - This panel shows or rerun commands that are entered at the command line.

Help Options

- The user can use help facilities to find out information about syntax and functionality and see working examples. There are three ways the user can get help in MATLAB:
  - text based help: Text based help can be obtained from the command prompt by typing help.
  - graphical help interface: This is a separate window and it is launched from the menu bar.
  - web based help : The command help can also be used to find out information about a specific function. For example: >> help cos

Input and Output

You input commands to MATLAB in the MATLAB Command Window. MATLAB returns output in two ways: Typically, text or numerical output is returned in the same Command Window, but graphical output appears in a separate graphics window.



Basic Syntax

MATLAB environment behaves like a super-complex calculator. You can enter commands at the >> command prompt.
MATLAB is an interpreted environment. In other words, you give a command and MATLAB executes it right away.

Hands on Practice

Type a valid expression, for example,
$5 + 5$
And press ENTER
ans = 10
3^2 % 3 raised to the power of 2
And press ENTER
ans = 9

7/0 % Divide by zero
And press ENTER
ans = Inf
warning: division by zero
$732 * 20.3$ And press ENTER
MATLAB executes it immediately and the result returned is: ans = 1.4860e+04

Predefined Variables

MATLAB provides some special expressions for some mathematical symbols, like pi for π, Inf for ∞, i (and j) for $\sqrt{-1}$ etc. **Nan** stands for 'not a number'.

Use of Semicolon (;) in MATLAB

Semicolon (;) indicates end of statement. However, if you want to suppress and hide the MATLAB output for an expression, add a semicolon after the expression.

For example,

$x = 3;$

$y = x + 5$

And press ENTER, MATLAB executes it immediately and the result returned is: $y = 8$

Adding Comments in MATLAB

The percent symbol (%) is used for indicating a comment line. For example,

$x = 9$ % assign the value 9 to x

You can also write a block of comments using the block comment operators % { and % }.

The MATLAB editor includes tools and context menu items to help you add, remove, or change the format of comments.

Commonly used Operators

MATLAB supports the following commonly used operators and special characters:

Operator	Purpose
+	Plus; addition operator.
-	Minus; subtraction operator.
*	Scalar and matrix multiplication operator.
.*	Array multiplication operator.
/	Right-division operator.
./	Array right-division operator.
:	generates regularly spaced elements and represents an entire row or column.
()	encloses function arguments and array indices; overrides precedence.

Naming Variables

Variable names consist of a letter followed by any number of letters, digits or underscore.
MATLAB is **case-sensitive.**
Variable names can be of any length, however, MATLAB uses only first N characters, where N is given by the function **namelengthmax.**

Saving Your Workspace

The **save** command is used for saving all the variables in the workspace, as a file with .mat extension, in the current directory.
For example,
save myfile
You can reload the file anytime later using the **load** command.
load myfile

Variables

In MATLAB environment, every variable is an array or matrix. You can assign variables in a simple way. For example,

$x = 3$ % defining x and initializing it with a value

$x = \text{sqrt}(16)$ % defining x and initializing it with an expression

Multiple assignments

a = 2; b = 7; c = a * b;

To Recall the variables

The **who** command displays all the variable names you have used.

The **whos** command displays little more about the variables.

The **clear** command deletes all (or the specified) variable(s) from the memory.

The format Command

Syntax is format *type*

type	**Result**	**Example**
`short`	Scaled fixed-point format with 5 digits	`3.1416`
`long`	Scaled fixed-point format with 15 digits for double and 7 digits for single	`3.14159265358979`
`short e`	Floating-point format with 5 digits	`3.1416e+000`
`long e`	Floating-point format with 15 digits for double and 7 digits for single	`3.141592653589793e+000`
`short g`	Best of fixed- or floating-point format with 5 digits	`3.1416`
`long g`	Best of fixed- or floating-point format with 15 digits for double and 7 digits for single	`3.14159265358979`
`short eng`	Engineering format with at least 5 digits and a power that is a multiple of 3	`3.1416e+000`
`long eng`	Engineering format with exactly 16 significant digits and a power that is a multiple of 3	`3.14159265358979e+000`
`bank`	Fixed dollars and cents	`3.14`

Creating vectors

A vector is a one-dimensional array of numbers. MATLAB allows creating two types of vectors:
Row vectors
Column vectors
Row vectors are created by enclosing the set of elements in square brackets, using space or comma to delimit the elements.
$r = [7 \quad 8 \quad 9 \quad 10 \quad 11]$
$t = [2, 3, 4, 5, 6];$
$res = r + t$
Column vectors are created by enclosing the set of elements in square brackets, using semicolon (;) to delimit the elements.
$c = [7; 8; 9; 10; 11]$

Creating Matrices

A matrix is a two-dimensional array of numbers.
In MATLAB, a matrix is created by entering each row as a sequence of space or comma separated elements, and end of a row is demarcated by a semicolon.
For example, let us create a 3-by-3 matrix as:
$m = [1 \quad 2 \quad 3; 4 \quad 5 \quad 6; 7 \quad 8 \quad 9]$

Commands

MATLAB is an interactive program for numerical computation and data visualization. You can enter a command by typing it at the MATLAB prompt '>>' on the Command Window.

Commands for Managing a Session

- **clc: Clears command window.**
- **clear: Removes variables from memory.**
- **exist: Checks for existence of file or variable.**
- **global: Declares variables to be global.**
- **help: Searches for a help topic.**
- **lookfor: Searches help entries for a keyword.**
- **quit: Stops MATLAB.**
- **who: Lists current variables.**
- **whos: Lists current variables (long display).**

Commands for Working with the System

- **cd: Changes current directory.**
- **date: Displays current date.**
- **delete: Deletes a file.**
- **diary: Switches on/off diary file recording.**
- **dir: Lists all files in current directory.**
- **load: Loads workspace variables from a file.**
- **path: Displays search path.**
- **pwd: Displays current directory.**
- **save: Saves workspace variables in a file.**
- **type: Displays contents of a file.**
- **what: Lists all MATLAB files in the current directory.**
- **wklread: Reads .wk1 spreadsheet file.**

Input and Output Commands

Command and purpose

- **disp:** Displays contents of an array or string.
- **fscanf:** Read formatted data from a file.
- **format:** Controls screen-display format.
- **fprintf:** Performs formatted writes to screen or file.
- **input:** Displays prompts and waits for input.
- **;** Suppresses screen printing.

Format code and purpose

Format Code	Description
`%d`	Integer format
`%e`	Scientific format with lowercase e
`%E`	Scientific format with uppercase E
`%f`	Decimal format
`%g`	The more compact of `%e` or `%f`

Control Code	Description
`\n`	Start new line
`\t`	Tab

M-files

MATLAB allows writing two kinds of program files.
Scripts- script files are program files with **.m extension**. In these files, you write series of commands, which you want to execute together. Scripts do not accept inputs and do not return any outputs. They operate on data in the workspace.
Functions- functions files are also program files with **.m extension**. Functions can accept inputs and return outputs. Internal variables are local to the function.

Introduction to script file

Scripts are the simplest kind of code file because they have no input or output arguments. They are useful for automating series of MATLAB commands, such as computations that you have to perform repeatedly from the command line or series of commands you have to refer.

Examples

Type the following code in the editor:

```
NoOfStudents =6000;
TeachingStaff=150;
NonTeaching Staff=20;
Total= NoOfStudents+Teaching Staff+NonTeachingStaff;
disp(Total);

a=5; b=7;
c=a+b
d=c+sin(b)
e=5*d
f=exp(-d)
```

Introduction to function environment and function handles

- The general syntax for creating a function is as follows:

```
function [outvar1, outvar2,..., outvarn]= function_name
(invar1,invar2,...invarn)

        block of code that uses your input variables invar1,invar2,invar3

        and returns the output variables, outvar1,outvar2,... outvarn.

end
```

- 'function' - keyword which tells MATLAB that you are about to define a function environment.
- 'function_name' - a user-defined name used to call the function to evaluate its commands

Example

To define the function $f(x) = sinx + lnx$ one can write the following lines of code:

```
function y = my_fun(x)

        y = sin(x) + log(x)

end
```

To evaluate the function at $x = 0.5$ you would run the command:

$$y = my_fun(0.5)$$

Introduction to function environment and function handles cont...

- MATLAB functions must always be defined at the end of your script file (where your program is being written) or in a separate script file.
- **For easy access, you should define your function environment in a separate script file**.
- It is important that you save the function script file with the same name as your function.
  ex: the environment for the function my_fun should be saved as my_fun.m

Recap: Function handle

Simple functions can be defined more quickly with what is called a function handle and defining as:

```
function_name = @(x_input) function_rule
```

- 'function_name' - user-defined name for the function.
- The @ symbol tells MATLAB you want to define a function handle with independent variable with name 'x_input'.
- 'function_rule' - operation you want the function to perform and it must be expressed in terms of the independent variable x_input.

Example

The function $f(x) = sinx + lnx$ can be defined using a function handle as follows:

```
my_fun = @(x) sin(x) + log(x)
```

To evaluate the function at $x = 0.5$ and store your result in the variable y, you would run the command:

$$y = my_fun(0.5)$$

Learning to displaying results in a Table

In this activity, you will learn how to create professional looking tables in MATLAB. The easiest way to display a table is to think of it as a collections of columns. The idea is to first store all the columns you seek to display in separate variables (as row vectors) and then use MATLAB's **fprintf** command to display table row by row, with each row containing an entry from each variable.

Example

Display the values of $f(x) = x^2 + lnx$ for $x = 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4$ in a Table with column headers x and $f(x)$.

- Step1: First create a vector to store the x values. There are three ways of doing this. Run the code to see that you obtain the same result for all cases. Use a semi colon to suppress the outputs if you don't want to see them.

```
%  List out all the values.
% This is useful when the list of input values is not evenly spaced
x1 = [0.5,1,1.5,2,2.5,3,3.5,4]
% You can also specify the vector as 'starting value: spacing: endvalue'.
% This is only possible for an evenly spaced list
x2 = 0.5:0.5:4
% You can also used linspace(start,end,# of points)
% to create a list of evenly spaced points.
x3 = linspace(0.5,4,8)
```

Example cont...

- Setp2: Next define your function to handle vector inputs. Use function handle or function environment to get the desired output.

```
my_fun = @(x) x.^2 + log(x)
```

Notice that in here there is a dot before the exponentiation operator. **This allows MATLAB to square vectors by squaring each entry of the vector. This is called vectorization in MATLAB. Use the same principle to divide (a./b) and multiple (a.*b) vector inputs a and b** when you want MATLAB to apply these operations to each entry of the vector.

Example cont...

- Step 3:Next, evaluate your function and store the outputs in a new variable.

```
y1 = my_fun(x1)
```

- Step 4:Now display the input values and output values in a table using **fprintf**

```
% Display the column headers
disp (' x          f(x)')

% Display the rest of the table
fprintf('%8.4f %8.4f \n',[x1;y1])
```

% (digit_space).(decimal_places)(display_type) (line_escape)

Example

Create a table to show values of the polynomial, $p(x) = x^3 - 4x^2 + 5x - 2$. Evaluate the function for x values between 0 and 2 at intervals of 0.25. Display two columns with headers $'x'$ and $'p(x)'$ and show your function values in floating point format with **5 decimal places.**

```
x4 = 0:0.25:2
fun_p1 = @(x) x.^3 - 4 * x.^2 + 5 * x - 2
p = fun_p1(x4)

disp('x          p(x)')
fprintf('%8.5f %8.5f \n',[x4;p])
```

Hand on Practice

1. Solve the following problems.
   a) $\frac{35.7.64-7^3}{45+5^2}$
   b) $\frac{3^7 log(76)}{7^3+546} + \sqrt[3]{910}$
   c) $\cos^2(\frac{5\pi}{6})\sin(\frac{7\pi}{8})^2 + \frac{\tan(\frac{\pi}{6}\ln 8)}{\sqrt{7}}$
2. Define the variable x as $x = 13.5$, then evaluate $x^3 + 5x^2 - 26.7x - 52$.
3. Calculate (by writing one command) the radius r of a sphere that has a volume of $350cm^3$. Once r is determined, use it to calculate the surface area of the sphere.

- Write a script in **MATLAB** to answer the questions.

4. Write a script to calculate the circumference of the circle. Use *fprintf* command to display the output.

Hand on Practice

5. Define the function $p(x) = x^3 - 4x^2 + 5x - 2$. Use a function handle in the first cell and a function environment in a separate script file. Choose different function names for each definition. Evaluate your function for $x = 1, 2, 10$. You should get an output $0, 0, 648$.
6. A cone shape paper cup is designed to have a volume of $250cm^3$. Determine the radius, r, of the base and the surface area, S, of the paper for cups with heights, h of $5, 6, 7, 8,$ and, 9 cm. The volume, V, and the surface area of the paper are given by:

$$V = \frac{1}{3}\pi r^2 h \qquad S = \pi r\sqrt{r^2 + h^2}$$

Operators

MATLAB allows the following types of elementary operations:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operations
- Set Operations

Arithmetic Operators

MATLAB allows two different types of arithmetic operations.

- Matrix arithmetic operations
- Array arithmetic operations

Functions for Arithmetic Operations

uplus(a), plus(a,b), uminus(a), minus(a,b), times(a,b), mtimes(a,b), rdivide(a,b), ldivide(a,b), mrdivide(A,B), mldivide(A,B), power(a,b), mpower(a,b), diff(X), diff(X,n), ...

Relation Operators

Example	Operator	Relationship
`x == 0`	`==`	Equal
`unit ~= 'm'`	`~=`	Not equal
`a < 0`	`<`	Less than
`s > t`	`>`	Greater than
`3.9 <= a/3`	`<=`	Less than or equal to
`r >= 0`	`>=`	Greater than or equal to

Relational Operators

Example:
a=100; b=200;
if $(a >= b)$
max $= a$
else max$= b$
end
Apart from the above mentioned relational operators, MATLAB provides the following commands/ functions used for the same purpose:
eq(a,b), ge(a,b), gt(a,b), le(a,b), lt(a,b), ne(a,b), isequal, isequaln

```
% comparing two values a=100;
b=200;
if (ge(a,b))
max = a
else
max = b
end
% comparing two different values
a=340;
b=520;
if (le(a,b))
disp('a is either less than or equal to b')
else
disp(' a is greater than b')
end
```

Set Operations

intersect(A,B), intersect(A,B,'rows'), ismember(A,B), ismember(A,B,'rows'), issorted(A), issorted(A,'rows'),setdiff(A,B), setdiff(A,B,'rows'), setxor, union, unique

```
a=[7 23 14 15 9 12 8 24 35]
b=[2 5 7 8 14 16 25 35 27]
u= union(a,b)
i=intersect(a,b)
s=setdiff(a,b)
```

Plotting and Graphics

- Adding Title, Labels. Grid Lines and Scaling on the graph
- Drawing Multiple Functions on the same graph
- Generating sub-plots
- Drawing bar charts, contours and three dimensional graphs

Plot a graph

To plot the graph of a function, you need to take the following steps:

- Define x, by specifying the **range of values** for the variable x, for which the function is to be plotted
- Define the function, $y = f(x)$
- Call the **plot** command, as **plot(x,y)**

Example

```
x= [0:5:100];
y=x;
plot(x,y)
```

Example

```
x= [1 2 3 4 5 6 7 8 9 10];
x=[-100:20:100];
y = x^2;
plot(x,y)
```

Change the code file a little, reduce the increment to 5: x=[-100:5:100];

```
y = x^2;
plot(x,y)
```

Adding Title, Labels, Grid Lines, and Scaling on the Graph

MATLAB allows you to add title, labels along the x-axis and y-axis, grid lines and also to adjust the axes to spruce up the graph.

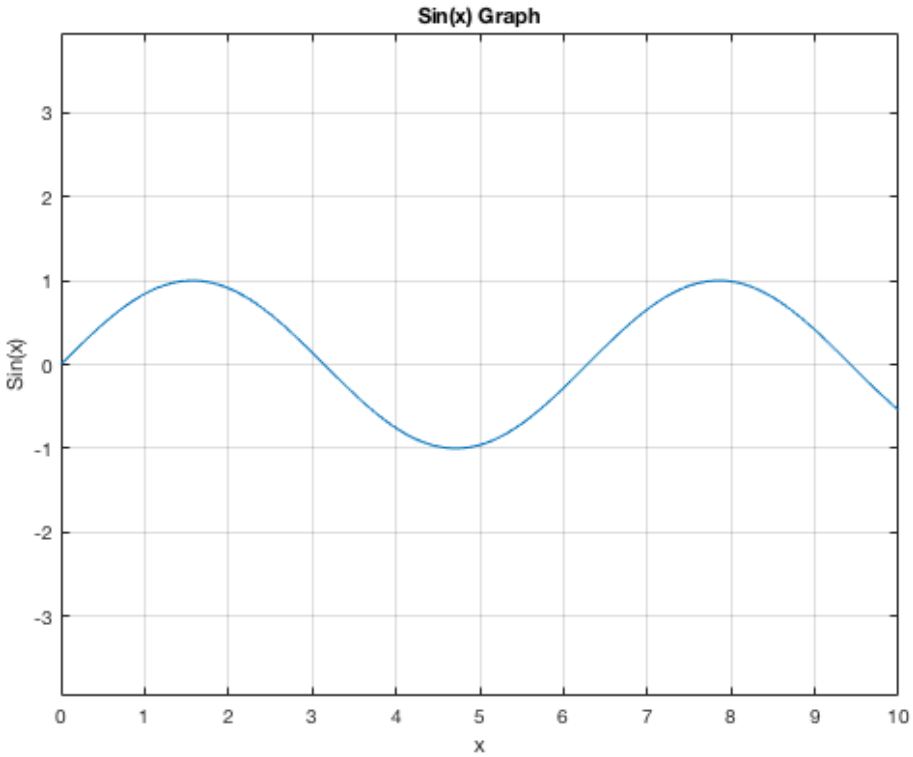
- The **xlabel** and **ylabel** commands generate labels along x-axis and y-axis.
- The **title** command allows you to put a title on the graph.
- The **grid on** command allows you to put the grid lines on the graph.
- The **axis equal** command allows generating the plot with the same scale factors and the spaces on both axes.
- The **axis square** command generates a square plot.

Example

x=[0:0.01:10];

$y = \sin x$;

plot(x,y), xlabel('x'), ylabel('Sin(x)'), title('Sin(x) Graph'), grid on, axis equal

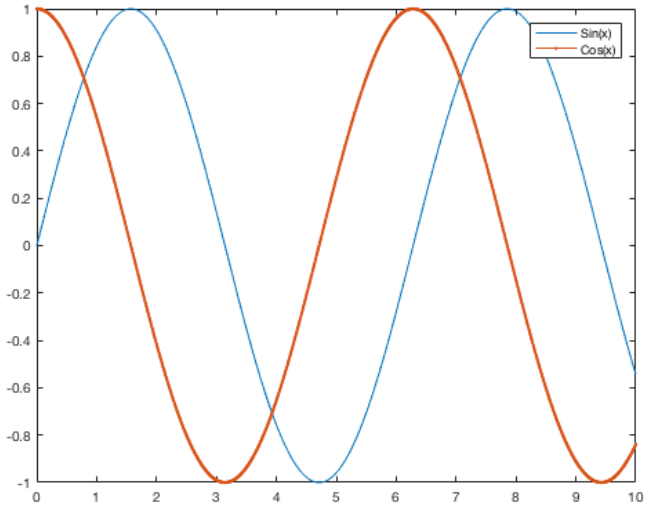


Drawing Multiple Functions on the Same Graph

You can draw multiple graphs on the same plot.

Example

x=[0:0.01:10];
$y = \sin x$;
$g = \cos x$;
plot(x, y, x, g, '.-'), legend('Sin(x)'),'Cos(x)')



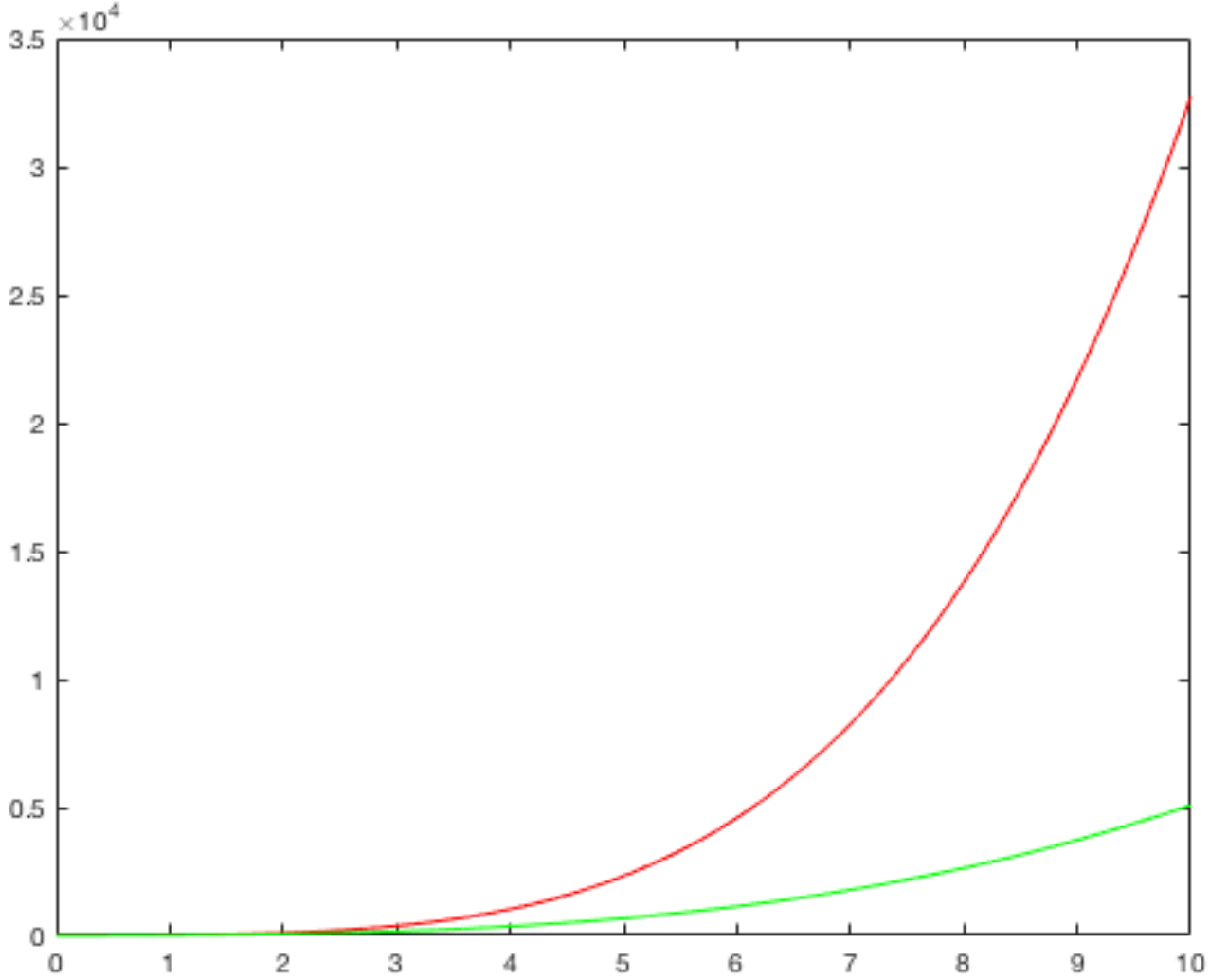
Setting Colors on Graph

MATLAB provides eight basic colour options for drawing graphs. The following table shows the colors and their codes:
w: White, **k**: Black, **b**: Blue, **r**: Red, **c**: Cyan, **g**: Green, **m**: Magenta, **y**: Yellow

Example

x=[-10:0.01:10];
$y = 3 * x^4 + 2 * x^3 + 7 * x^2 + 2 * x + 9;$
$g = 5 * x^3 + 9 * x + 2;$
plot(x, y, 'r', x, g, 'g')

Example cont...



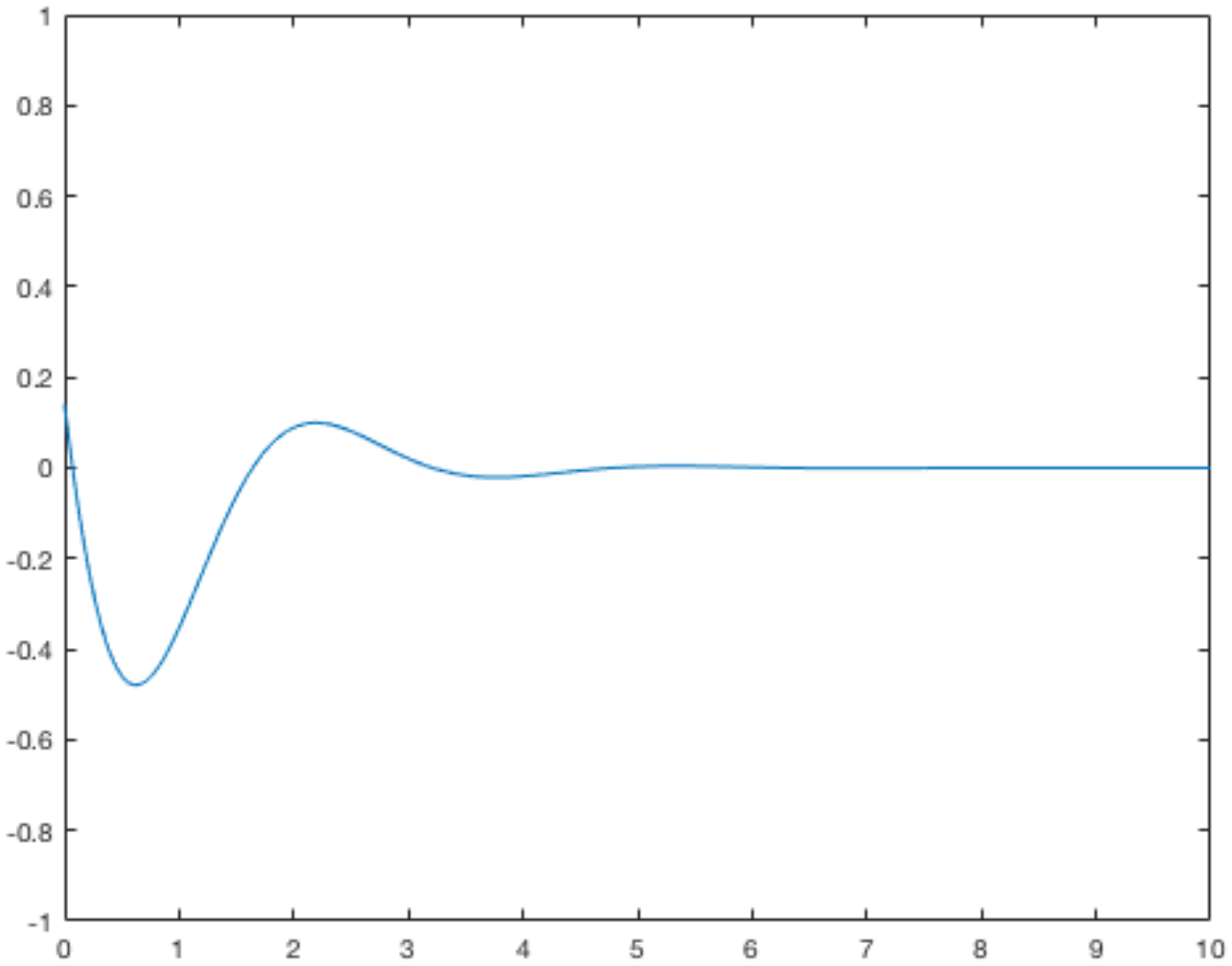
Setting Axis Scales

The **axis** command allows you to set the axis scales. You can provide minimum and maximum values for x and y axes using the axis command in the following way: *axis*([*xminxmaxyminymax*])

Example

```
x= [0: 0.01: 10];
y= exp(-x).*sin(2*x+3);
plot(x,y), axis( [0 10 -1 1] )
```

Example cont...



Generating Sub-Plots

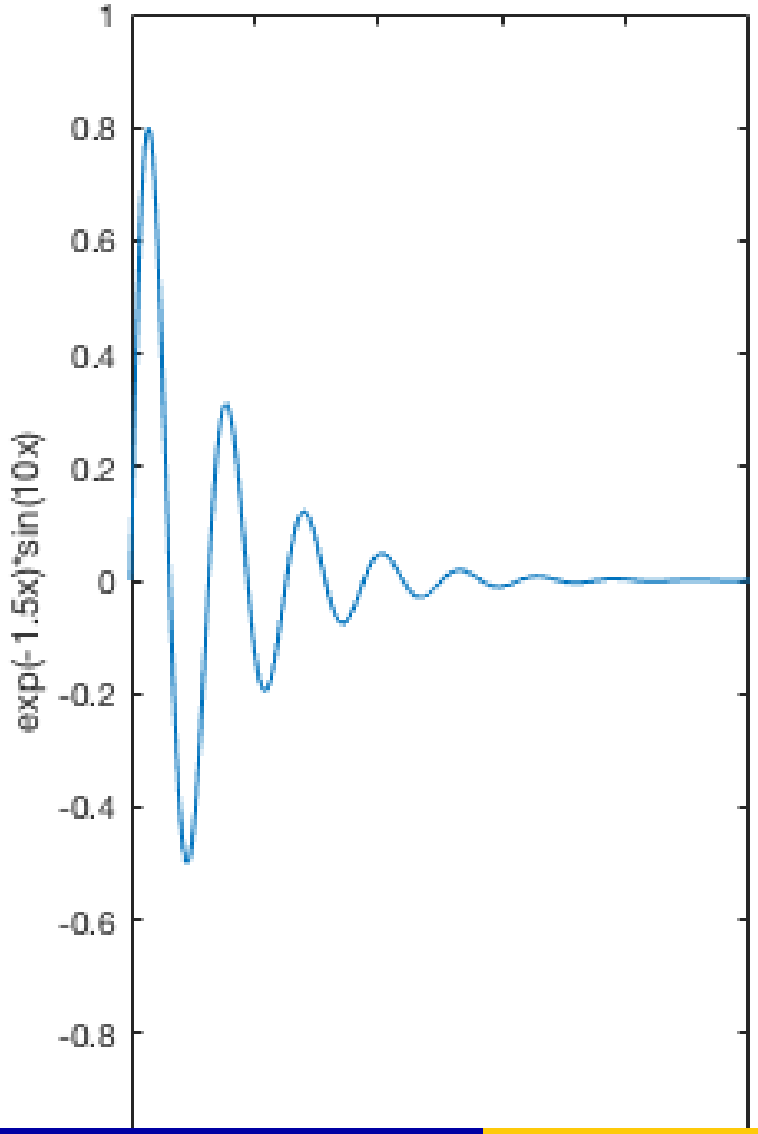
When you create an array of plots in the same figure, each of these plots is called a subplot. The **subplot** command is used for creating subplots. Syntax for the command is: subplot(m, n, p) where, *m* and *n* are the number of rows and columns of the plot array and *p* specifies where to put a particular plot. Each plot created with the subplot command can have its own characteristics.

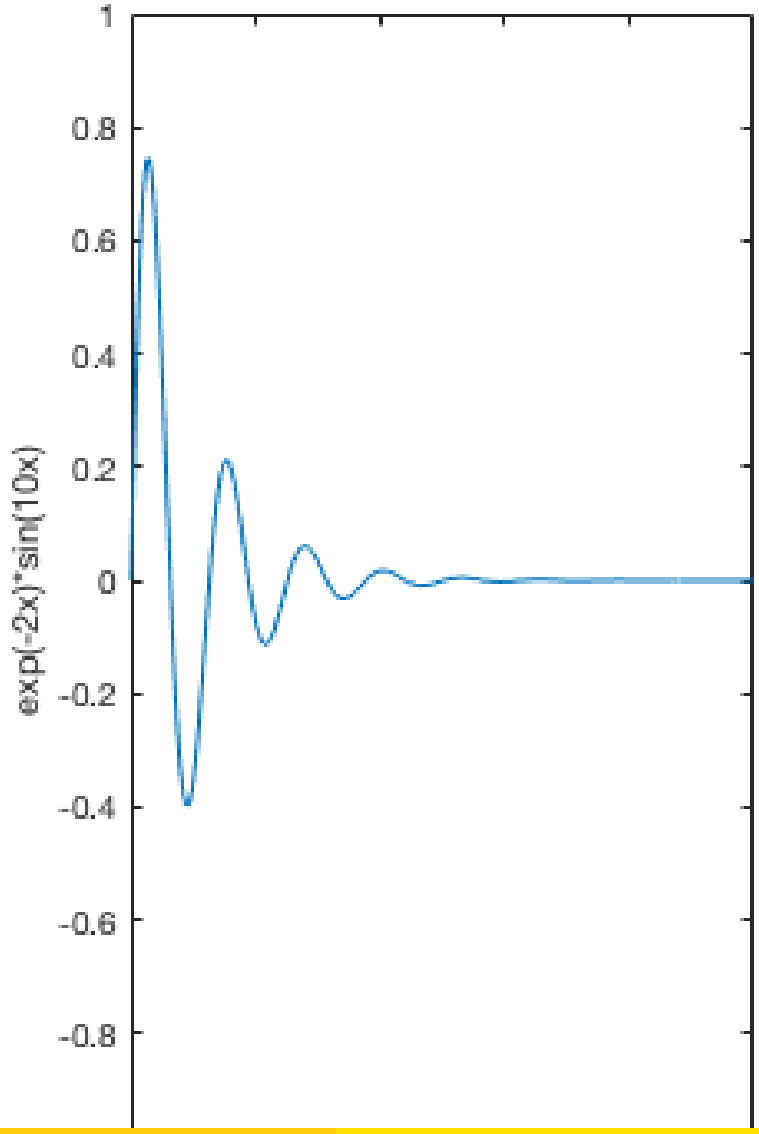
Example

$x = [0 : 0.01 : 5];$
$y = exp(-1.5 * x). * sin(10 * x);$

```
subplot(1,2,1)
plot(x,y), xlabel('x'), ylabel('exp(-1.5x)*sin(10x)'), axis([0 5 -1 1])
y= exp(-2*x).*sin(10*x);
subplot(1,2,2)
plot(x,y), xlabel('x'), ylabel('exp(-2x)*sin(10x)'), axis([0 5 -1 1])
```

Example cont...





Graphics

This section will continue exploring the plotting and graphics capabilities of MATLAB.

- Drawing bar charts
- Drawing contours
- Three dimensional plots

Drawing bar charts

The **bar** command draws a two dimensional bar chart.

Example

Let us have an imaginary classroom with 10 students. We know the percent of marks obtained by these students are 75, 58, 90, 87, 50, 85, 92, 75, 60 and 95. We will draw the bar chart for this data.

Bar Charts

Create a script file and type the following code:

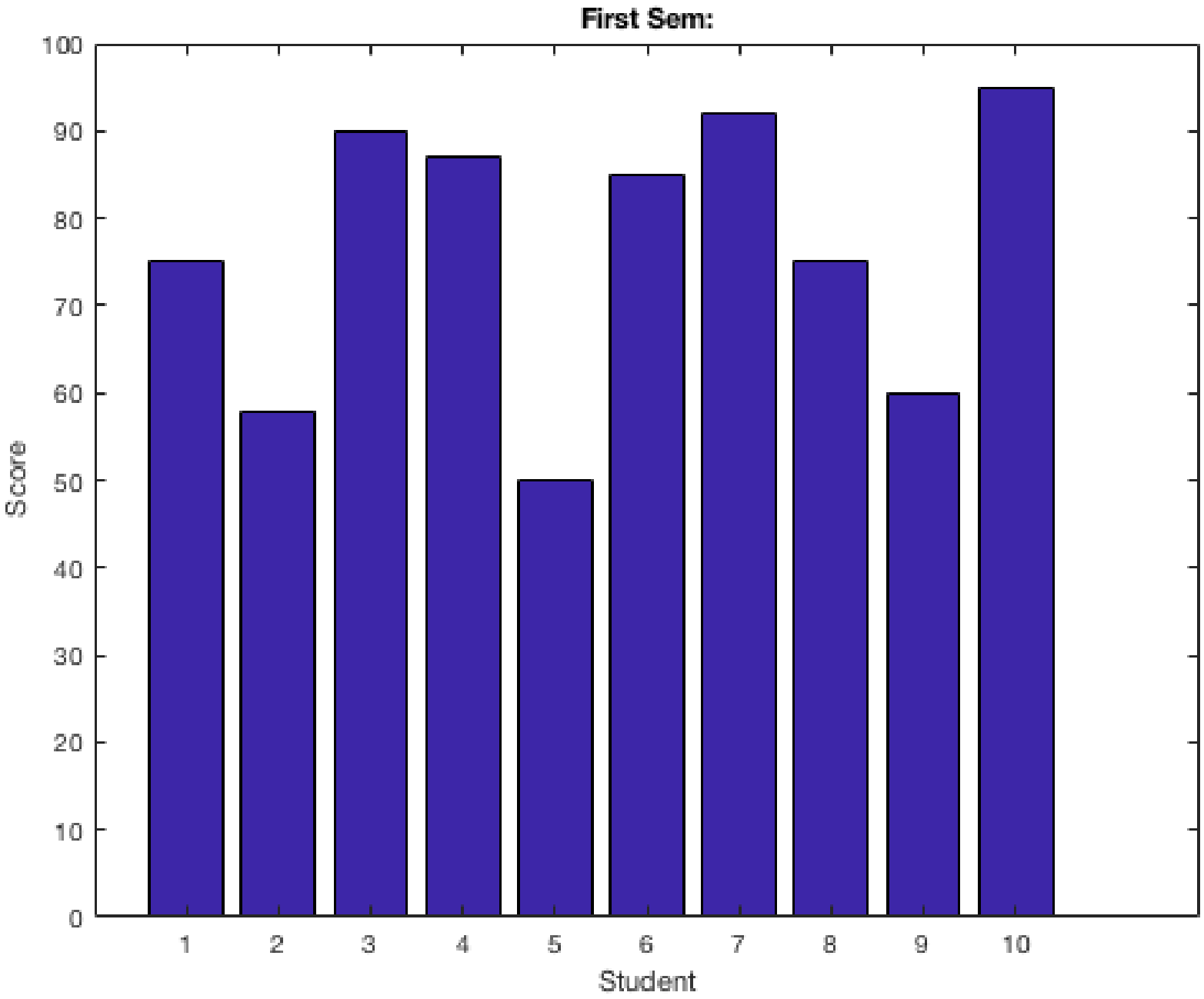
$x = [1 : 10]$;

$y = [75, 58, 90, 87, 50, 85, 92, 75, 60, 95]$;

bar(x,y), xlabel('Student'), ylabel('Score')

title('First Sem:')

print -deps graph.eps


First Sem:
100
90
80
70
60
50
40
30
20
10
0
Score
1
2
3
4
5
6
7
8
9
10
Student

Drawing Contours

- A contour line of a function of two variables is a curve along which the function has a constant value. Contour lines are used for creating contour maps by joining points of equal elevation above a given level, such as mean sea level.
- MATLAB provides a **contour** function for drawing contour maps.

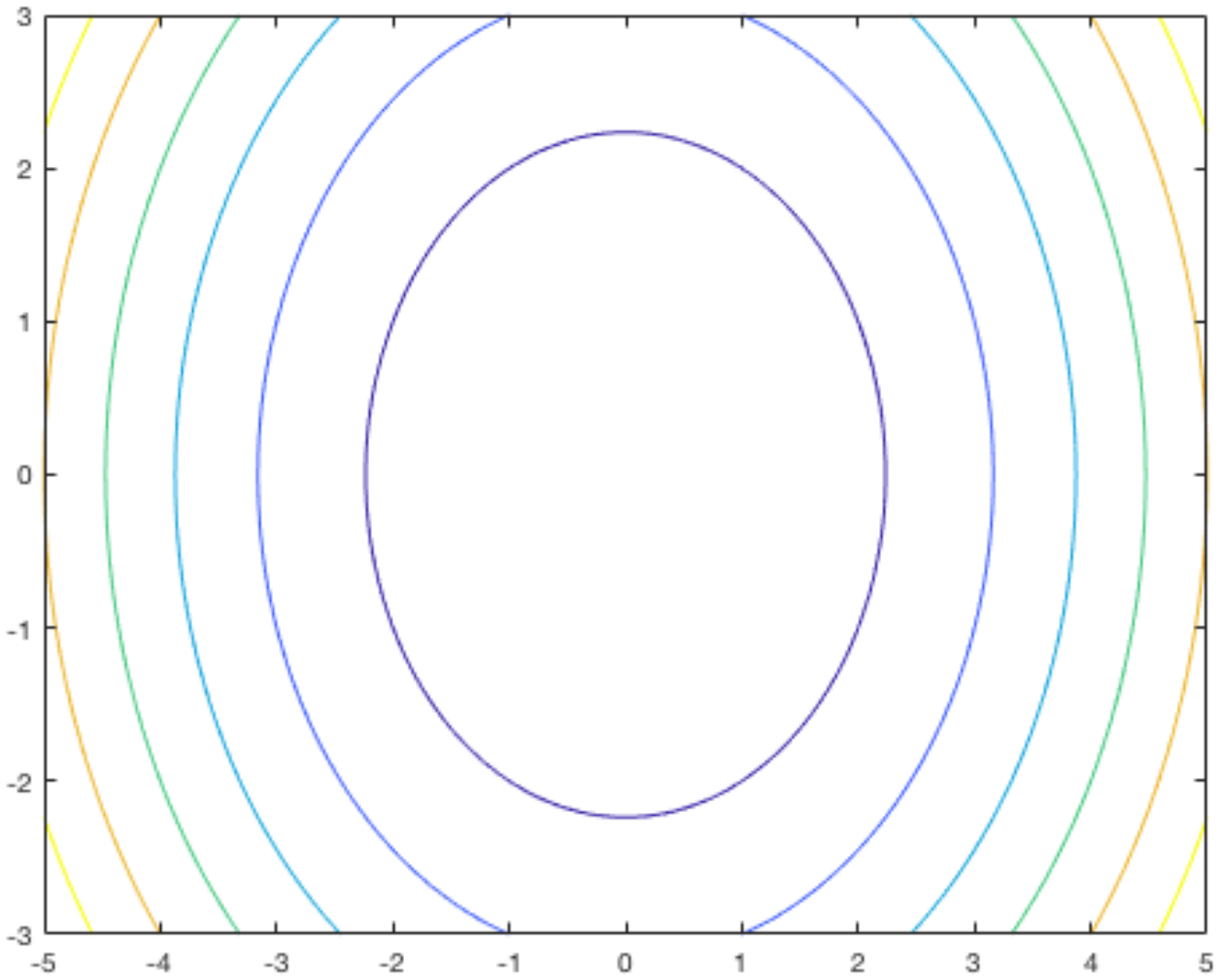
Example

Let us generate a contour map that shows the contour lines for a given function $g = f(x, y)$. This function has two variables. So, we will have to generate two independent variables, i.e., two data sets x and y. This is done by calling the **meshgrid** command.
The **meshgrid** command is used for generating a matrix of elements that give the range over x and y along with the specification of increment in each case.

Drawing Contours

$[x, y] = meshgrid(-5 : 0.1 : 5, -3 : 0.1 : 3);$
Lastly, we need to assign the function. Let our function be: $x^2 + y^2$
Create a script file and type following code:
$[x, y] = meshgrid(-5 : 0.1 : 5, -3 : 0.1 : 3); \%independentvariables$
$g = x^2 + y^2$; % our function
contour(x,y,g) % call the contour function

Drawing Contours



Drawing Contours

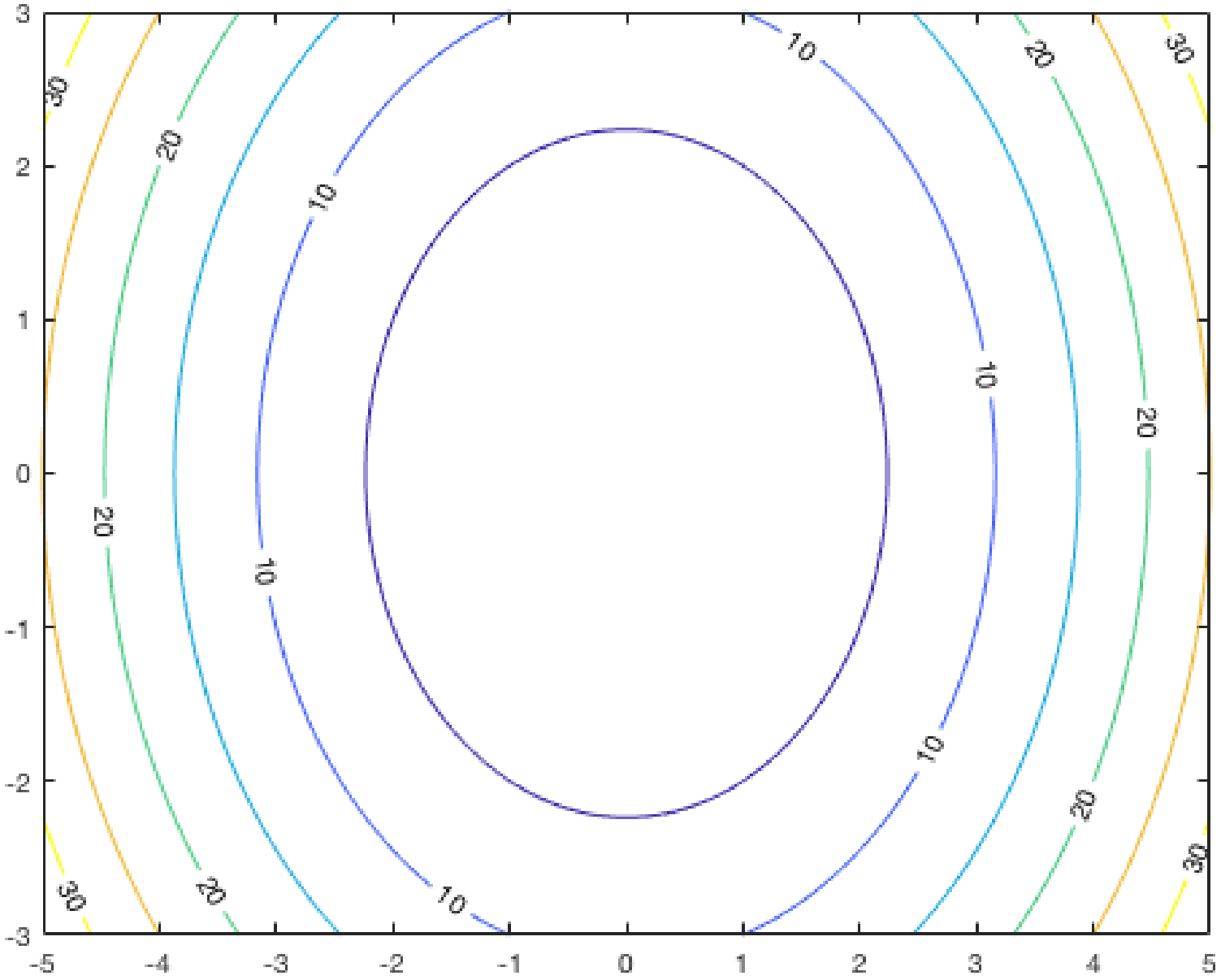
Let us modify the code a little to spruce up the map:

$[x, y] = meshgrid(-5 : 0.1 : 5, -3 : 0.1 : 3); \%independentvariables$

$g = x^2 + y^2$; % our function

$[C, h] = contour(x, y, g)$; % call the contour function

set(h, 'ShowText', 'on', 'TextStep', get(h,'LevelStep')*2)

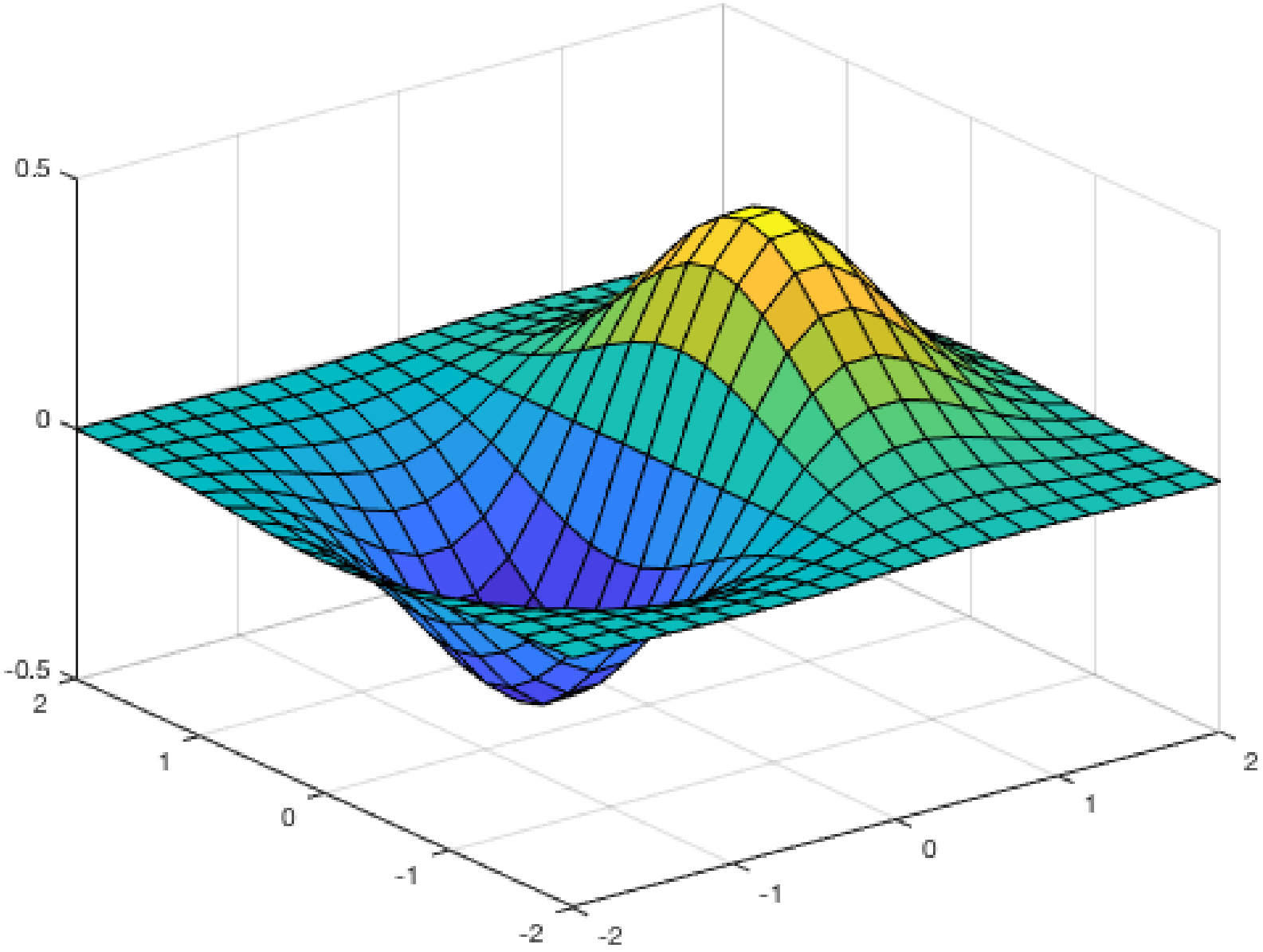

3
2
1
0
-1
-2
-3
-5
-4
-3
-2
-1
0
1
2
3
4
5
10
20
30

Three dimensional Plots

Three-dimensional plots basically display a surface defined by a function in two variables, $g = f(x, y)$. As before, to define g, we first create a set of (x, y) points over the domain of the function using the **meshgrid** command. Next, we assign the function itself. Finally, we use the **surf** command to create a surface plot.

Example

Let us create a 3D surface map for the function $g = xe^{-(x^2+y^2)}$ Create a script file and type the following code: [x,y] = meshgrid(-2:.2:2);
$g = x. * exp(-x^2 - y^2)$;
surf(x, y, g)
print -deps graph.eps


0.5
0
-0.5
2
1
0
-1
-2
-2
-1
0
1
2

You can also use the **mesh** command to generate a three-dimensional surface. However, the **surf** command displays both the connecting lines and the faces of the surface in color, whereas the **mesh** command creates a wireframe surface with colored lines connecting the defining points.